

Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Корутины

- suspend function
- scope
- builders
- exceptions
- flow

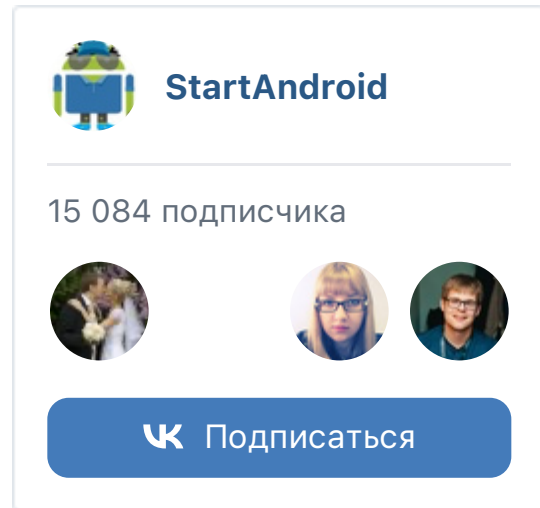
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

[Яндекс](#)
410011180491924

[WebMoney](#)
R248743991365
Z551306702056

[Paypal](#)

Урок 12. Multiscope

21 июня 2021

В этом уроке рассмотрим возможность добавления компонентам нескольких scope аннотаций.

Продолжаем тему scope. Рассмотрим немного более сложный случай, чем в прошлом уроке.

Пусть у нас есть класс UiHelper. Он нужен, чтобы, например, показывать Toast или диалоги.

```
1 class UiHelper(activity: Activity) {
2     // ...
3 }
```

На вход он требует Activity.

И у нас есть OrderActivity и UserActivity со своими сабкомпонентами OrderComponent и UserComponent. В этих Activity мы хотели бы получать синглтон UiHelper от их сабкомпонентов.

Давайте посмотрим, как это можно сделать.

Provides

Достаточно легко это реализуется с помощью стандартной схемы, когда сабкомпонент достаёт объект из своего модуля и с помощью scope аннотации делает синглтон.

Выглядит это так.

Сабкомпонент OrderComponent использует scope аннотацию OrderScope и модуль OrderModule:

```
1 @OrderScope
2 @Subcomponent(modules = [OrderModule::class])
3 interface OrderComponent {
4     // ...
5 }
```

В модуле мы прописываем создание объекта UiHelper и добавляем scope аннотацию OrderScope.

```
1 @Module
2 class OrderModule {
3
4     @OrderScope
5     @Provides
6     fun provideUiHelper(activity: Activity): UiHelper {
7         return UiHelper(activity)
8     }
9
10 }
```

В итоге сабкомпонент OrderComponent будет создавать свой синглтон этого объекта. И мы получим его в OrderActivity.

Ровно то же самое делаем для UserComponent.

```
1 @UserScope
2 @Subcomponent(modules = [UserModule::class])
3 interface UserComponent {
4     // ...
5 }
```

У него есть UserScope и UserModule.

В модуле создаем UiHelper и указываем scope аннотацию

```
1 @Module
2 class UserModule {
3
4     @UserScope
5     @Provides
6     fun provideUiHelper(activity: Activity): UiHelper {
7         return UiHelper(activity)
8     }
9
10 }
```

В итоге сабкомпонент UserComponent будет создавать свой синглтон этого объекта. И мы получим его в UserActivity.

Два сабкомпонента создают свои синглтоны объекта UiHelper. В целом тут ничего нового для нас не было.

Давайте посмотрим, можно ли то же самое сделать, если объект создается не в модуле, а Inject конструктором.

Inject конструктор

У нас есть два сабкомпонента со своими scope аннотациями. И они хотят создавать синглтон этого объекта:

```
1 class UiHelper @Inject constructor(activity: Activity) {
2     // ...
3 }
```

Как мы помним, для получения синглтона объекта нам надо добавить к нему scope аннотацию сабкомпонента. И т.к. сабкомпонента у нас два, то и добавлять надо будет две scope аннотации:

```
1 @OrderScope
2 @UserScope
3 class UiHelper @Inject constructor(activity: Activity) {
4     // ...
5 }
```

Теперь, по идее, каждый из сабкомпонентов OrderComponent и UserComponent будет видеть у объекта свою аннотацию и создавать синглтон.

Но этот код не скомпилируется. Даггер скажет нам, что к объекту можно применить только одну scope аннотацию.

Есть другое решение. Мы для нескольких компонентов (или сабкомпонентов) можем создать одну общую scope аннотацию (в дополнение к уже существующим). И эту scope аннотацию мы и повесим на объект, чтобы получать синглтон.

Создаем новую scope аннотацию ActivityScope:

```
1 @Scope
2 @Retention(AnnotationRetention.RUNTIME)
3 annotation class ActivityScope
```

Почему мы назвали ее именно так? Потому что мы собираемся повесить ее на сабкомпоненты, время жизни которых совпадает с временем жизни их Activity. Название этой аннотации однозначно покажет, что время жизни объекта совпадает с временем жизни Activity, в котором он используется как синглтон.

Добавим эту аннотацию к сабкомпонентам:

OrderComponent:

```
1 @ActivityScope
2 @OrderScope
3 @Subcomponent(modules = [OrderModule::class])
4 interface OrderComponent
```

UserComponent:

```
1 @ActivityScope
2 @UserScope
3 @Subcomponent(modules = [UserModule::class])
4 interface UserComponent
```

У сабкомпонентов теперь по две scope аннотации. Это не запрещено.

Добавим scope аннотацию к UiHelper:

```
1 @ActivityScope
2 class UiHelper @Inject constructor(activity: Activity) {
3     // ...
4 }
```

Теперь сабкомпоненты OrderComponent и UserComponent будут создавать синглтоны UiHelper, потому что видят у этого объекта одну из своих scope аннотаций (ActivityScope). Причем каждый из этих сабкомпонентов будет создавать свой синглтон.

Если нам надо, чтобы эти сабкомпоненты имели общий синглтон, то к UiHelper надо поставить аннотацию AppScope. Тогда AppComponent будет создавать синглтон этого объекта и предоставлять его обоим сабкомпонентам.

Назад

Вперёд

Комментарии

- - **Дмитрий** 25.06.2021 16:16
здесь не хватает примера как на уроках предыдущих - что теперь будет за объект на разных активности - ведь у UserActivity и OrderActivity должны свои ссылки приходить в конструктор UiHelper - по идее объекты должны быть разными?

Ответить

RE: - - Dmitry Vinogradov 26.06.2021 23:28
Да, каждый сабкомпонент будет создавать свой синглтон. Дополнил текст урока.

Ответить

Scope аннотации - **Евгений** 11.09.2022 20:13
А может в примере с inject конструктором orderscope и userscope аннотации в сабкомпонентах вообще убрать можно?

Ответить

RE: Scope аннотации - **Dmitry Vinogradov** 20.09.2022 19:04
Тогда мы не получим синглтон объекта

Ответить

Scope аннотации - **Евгений** 23.09.2022 19:49
Ну как же? По идее за счёт аннотации @ActivityScope должны получить, да и вы пишете так - "И эту scope аннотацию мы и повесим на объект, чтобы получать синглтон". Мне кажется это будет один и тот же объект, если оба активности запросят его, а существовать он будет до закрытия последней из этих активности. Но я в ide не набирал, может даггер не позволит так сделать

Ответить

RE: Scope аннотации - **Dmitry Vinogradov** 30.09.2022 14:43
А, я не понял вопрос. Конкретно в этом примере - да. Аннотации @OrderScope и @UserScope не нужны для UiHelper. Предполагается, что эти аннотации используются для создания синглтонов других объектов.

Ответить

Причем каждый из этих сабкомпонентов будет создавать свой синглтон - **Даниил** 14.11.2023 08:59
Тогда получается, что в последнем случае с inject, если обе активности будут жить в одно и то же время(например, одна стартует из другой), то у обоих будет один и тот же синглтон UiHelper? А смысл же был урока в том, чтобы у каждого был свой, отдельный синглтон? Ну с другой стороны если эти активности не живут одновременно, то это не проблема

Ответить

RE: Причем каждый из этих сабкомпонентов будет создавать свой синглтон - **Dmitry Vinogradov** 04.12.2023 20:37
Порядок вызовов Activity не повлияет на создание их сабкомпонентами синглтонов. Каждое Activity создает свой сабкомпонент и получает свой синглтон.

Ответить

🔄 Обновить список комментариев
📄 RSS лента комментариев этой записи

Добавить комментарий

Тема

😄 😊 😍 😂 😇 😈 😉 😊 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😽

Отправить

.JComments

